

AI Assisted coding

Assignment-3.1

A.Charani

2303A52108

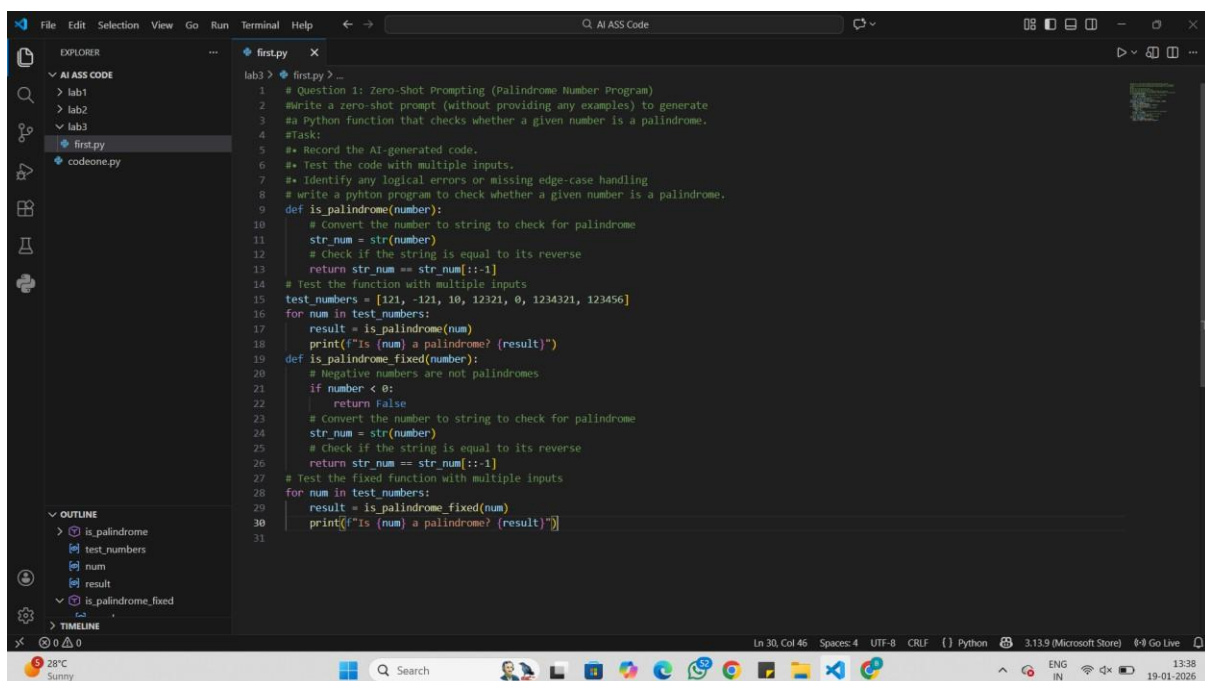
Batch-40

Question 1: Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

- Record the AI-generated code.
- Test the code with multiple inputs.
- Identify any logical errors or missing edge-case handling.



The screenshot shows a VS Code editor window with a file named 'first.py'. The code is a Python script that defines two functions to check if a number is a palindrome. The first function, 'is_palindrome', converts the number to a string and checks if it is equal to its reverse. The second function, 'is_palindrome_fixed', handles negative numbers by returning False if the number is less than 0. Both functions are tested with a list of numbers: [121, -121, 10, 12321, 0, 1234321, 123456]. The output of the script is printed for each number in the list.

```
lab3 > first.py > ...
1 # Question 1: Zero-Shot Prompting (Palindrome Number Program)
2 # Write a zero-shot prompt (without providing any examples) to generate
3 # a Python function that checks whether a given number is a palindrome.
4 # Task:
5 # • Record the AI-generated code.
6 # • Test the code with multiple inputs.
7 # • Identify any logical errors or missing edge-case handling.
8 # Write a Python program to check whether a given number is a palindrome.
9
10 def is_palindrome(number):
11     # Convert the number to string to check for palindrome
12     str_num = str(number)
13     # Check if the string is equal to its reverse
14     return str_num == str_num[::-1]
15
16 # Test the function with multiple inputs
17 test_numbers = [121, -121, 10, 12321, 0, 1234321, 123456]
18 for num in test_numbers:
19     result = is_palindrome(num)
20     print(f'Is {num} a palindrome? {result}')
21
22 def is_palindrome_fixed(number):
23     # Negative numbers are not palindromes
24     if number < 0:
25         return False
26     # Convert the number to string to check for palindrome
27     str_num = str(number)
28     # Check if the string is equal to its reverse
29     return str_num == str_num[::-1]
30
31 # Test the fixed function with multiple inputs
32 for num in test_numbers:
33     result = is_palindrome_fixed(num)
34     print(f'Is {num} a palindrome? {result}')
```

Output:

The screenshot shows a VS Code editor with a file named `first.py` open. The code is a Python script for checking if a number is a palindrome. The terminal output shows the script being executed with various test cases. The output is as follows:

```
lab3 > first.py > ...
1 # Question 1: Zero-Shot Prompting (Palindrome Number Program)
2 # Write a zero-shot prompt (without providing any examples) to generate
3
4 Is 1234321 a palindrome? True
5 Is 121 a palindrome? True
6 Is -121 a palindrome? False
7 Is 10 a palindrome? False
8 Is 12321 a palindrome? True
9 Is 0 a palindrome? True
10 Is 1234321 a palindrome? True
11 Is 123456 a palindrome? False
12 Is 121 a palindrome? True
13 Is -121 a palindrome? False
14 Is 10 a palindrome? False
15 Is 12321 a palindrome? True
16 Is 0 a palindrome? True
17 Is 1234321 a palindrome? True
18 Is 123456 a palindrome? False
19 PS C:\Users\tejas\College\AI ASS Code>
PS C:\Users\tejas\College\AI ASS Code> 
```

Analysis:

Works correctly for basic positive numbers

Negative numbers fail due to string behavior, not real logic

No input type checking is done

Relies only on string conversion

Suitable only for simple or beginner-level tasks

Question 2: One-Shot Prompting (Factorial Calculation)

Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Example:

Input: 5 → Output: 120

Task:

- Compare the generated code with a zero-shot solution.

- Examine improvements in clarity and correctness.

Output:

The screenshot shows a VS Code editor with a file named 'Sec.py' open. The code defines a factorial function and tests it with multiple inputs. The terminal output shows the results of these tests.

```

lab3 > Sec.py
1 #write a python program to print factorial of a number.
2 #Input: 5
3 #Output: 120
4 def factorial(n):
5     if n < 0:
6         return "Factorial is not defined for negative numbers"
7     elif n == 0 or n == 1:
8         return 1
9     else:
10        result = 1
11        for i in range(2, n + 1):
12            result *= i
13        return result
14 # Test the factorial function with multiple inputs
15 test_values = [5, 0, 1, 7, -3]
16 for val in test_values:
17     print(f"Factorial of {val} is: {factorial(val)}")
18 # Fixed version of the factorial function with edge case handling

```

Terminal Output:

```

PS C:\Users\tejas\College\AI ASS Code> & C:/Users/tejas/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/tejas/College/AI ASS Code/lab3/Sec.py"
Factorial of 5 is: 120
Factorial of 0 is: 1
Factorial of 1 is: 1
Factorial of 7 is: 5040
Factorial of -3 is: Factorial is not defined for negative numbers
PS C:\Users\tejas\College\AI ASS Code>

```

Analysis:

One-shot code clearly handles the base case ($0! = 1$)

Zero-shot version misses explicit handling of zero

One-shot solution is easier to understand and more structured

One-shot result is mathematically more correct

Example helps the AI generate safer and clearer logic.

Question 3: Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

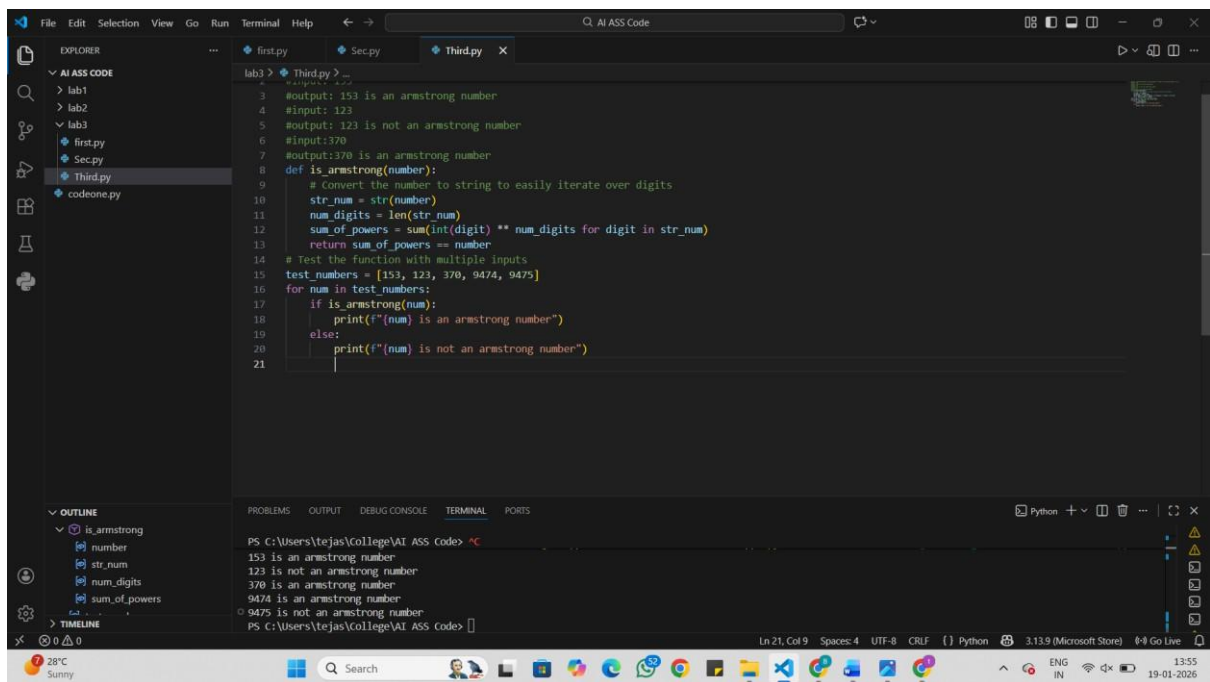
Examples:

- Input: 153 → Output: Armstrong Number

- Input: 370 → Output: Armstrong Number
- Input: 123 → Output: Not an Armstrong Number

Task:

- Analyze how multiple examples influence code structure and accuracy.
- Test the function with boundary values and invalid inputs.



```

lab3 > Third.py > ...
1 #input: 153
2 #output: 153 is an armstrong number
3 #input: 123
4 #output: 123 is not an armstrong number
5 #input: 370
6 #output: 370 is an armstrong number
7
8 def is_armstrong(number):
9     # Convert the number to string to easily iterate over digits
10    str_num = str(number)
11    num_digits = len(str_num)
12    sum_of_powers = sum(int(digit) ** num_digits for digit in str_num)
13    return sum_of_powers == number
14
15 # Test the function with multiple inputs
16 test_numbers = [153, 123, 370, 9474, 9475]
17 for num in test_numbers:
18     if is_armstrong(num):
19         print(f"{num} is an armstrong number")
20     else:
21         print(f"{num} is not an armstrong number")

```

Terminal Output:

```

PS C:\Users\tejas\College\AI ASS Code> ^C
153 is an armstrong number
123 is not an armstrong number
370 is an armstrong number
9474 is an armstrong number
9475 is not an armstrong number
PS C:\Users\tejas\College\AI ASS Code>

```

Analysis:

Giving examples helps the AI understand what kind of answer is expected.

Multiple examples make the code cleaner and more logical.

Showing both correct and incorrect cases avoids confusion.

Testing small numbers like 0 and 1 ensures the function works properly.

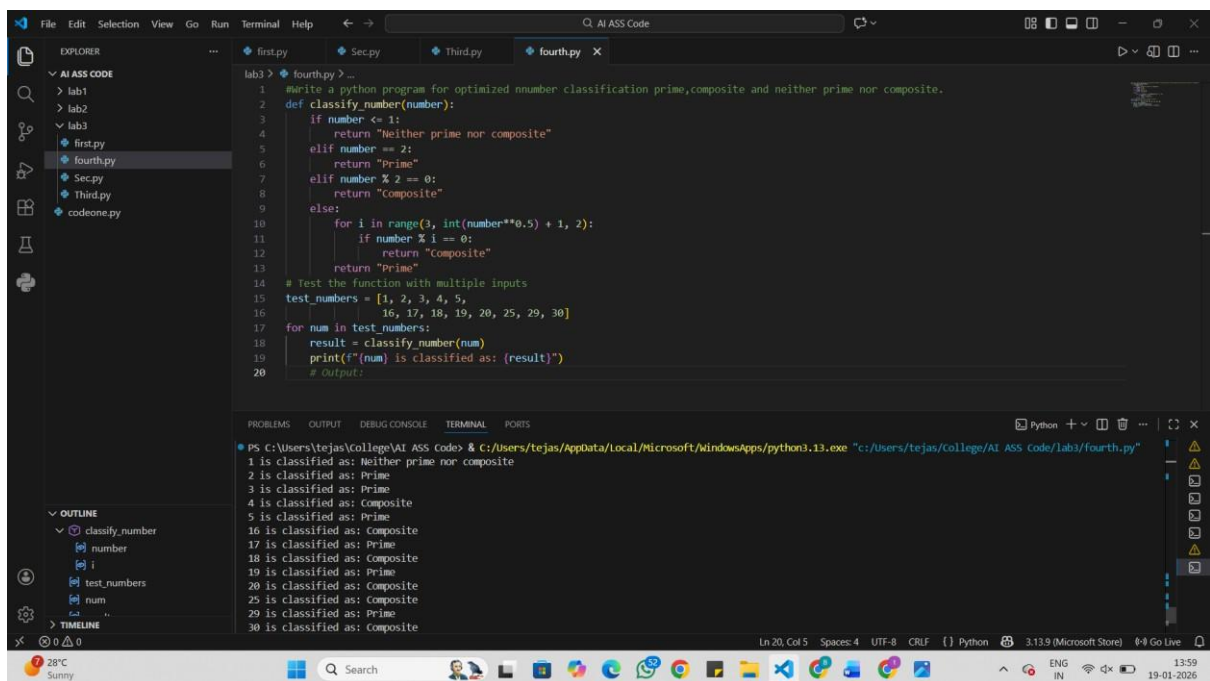
Checking wrong inputs makes the program safer and more reliable.

Question 4: Context-Managed Prompting (Optimized Number Classification)

Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.
- Compare the output with earlier prompting strategies.



The screenshot shows a Visual Studio Code editor with a Python file named `fourth.py` open. The code defines a function `classify_number(number)` that classifies a number as prime, composite, or neither. It includes input validation for non-positive numbers and an optimized loop for checking divisibility. The code also tests the function with a list of numbers and prints the results.

```
1 #write a python program for optimized number classification prime,composite and neither prime nor composite.
2 def classify_number(number):
3     if number <= 1:
4         return "Neither prime nor composite"
5     elif number == 2:
6         return "Prime"
7     elif number % 2 == 0:
8         return "Composite"
9     else:
10        for i in range(3, int(number**0.5) + 1, 2):
11            if number % i == 0:
12                return "Composite"
13        return "Prime"
14 # Test the function with multiple inputs
15 test_numbers = [1, 2, 3, 4, 5,
16                16, 17, 18, 19, 20, 25, 29, 30]
17 for num in test_numbers:
18     result = classify_number(num)
19     print(f"{num} is classified as: {result}")
20 # Output:
```

The terminal output shows the results of the classification for each number in the test list:

```
PS C:\Users\tejas\College\AI ASS Code> & C:/Users/tejas/AppData/Local/Microsoft/Windows/Apps/python3.13.exe "c:/Users/tejas/College/AI ASS Code/labs/fourth.py"
1 is classified as: Neither prime nor composite
2 is classified as: Prime
3 is classified as: Prime
4 is classified as: Composite
5 is classified as: Prime
16 is classified as: Composite
17 is classified as: Prime
18 is classified as: Composite
19 is classified as: Prime
20 is classified as: Composite
25 is classified as: Composite
29 is classified as: Prime
30 is classified as: Composite
```

Analysis:

Clear instructions help the AI understand exactly what is needed.

Input validation avoids crashes from wrong inputs.

Efficient logic makes the program faster and smarter.

The AI performs better than with simple prompts.

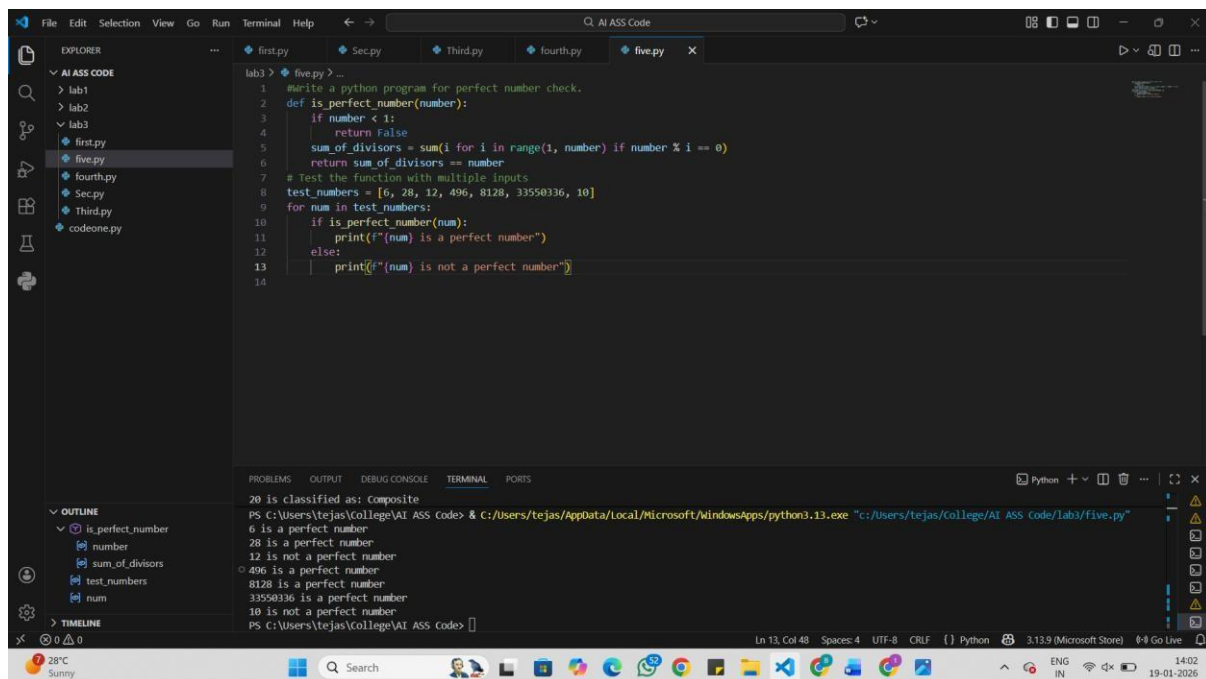
Results are clearer and more reliable.

Question 5: Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

- Record the AI-generated code.
- Test the program with multiple inputs.
- Identify any missing conditions or inefficiencies in the logic.



The screenshot shows a Visual Studio Code editor with a Python file named 'five.py' open. The code defines a function 'is_perfect_number' and tests it with a list of numbers. The terminal output shows the results of these tests.

```
1 #write a python program for perfect number check.
2 def is_perfect_number(number):
3     if number < 1:
4         return False
5     sum_of_divisors = sum(i for i in range(1, number) if number % i == 0)
6     return sum_of_divisors == number
7 # Test the function with multiple inputs
8 test_numbers = [6, 28, 12, 496, 8128, 33550336, 10]
9 for num in test_numbers:
10     if is_perfect_number(num):
11         print(f"{num} is a perfect number")
12     else:
13         print(f"{num} is not a perfect number")
14
```

Terminal Output:

```
PS C:\Users\tejas\College\AI ASS Code> & C:/Users/tejas/AppData/Local/Microsoft/WindowsApps/python3.13.exe "C:/Users/tejas/College/AI ASS Code/Lab3/Five.py"
20 is classified as: composite
6 is a perfect number
28 is a perfect number
12 is not a perfect number
496 is a perfect number
8128 is a perfect number
33550336 is a perfect number
10 is not a perfect number
PS C:\Users\tejas\College\AI ASS Code>
```

Analysis:

The AI works only with instructions, no examples.

The code usually works but may miss some cases.

Testing with different numbers shows if it's correct.

The logic may be slower without optimization.

Extra checks improve accuracy.

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

- Input: 8 → Output: Even
- Input: 15 → Output: Odd
- Input: 0 → Output: Even

Task:

- Analyze how examples improve input handling and output clarity.
- Test the program with negative numbers and non-integer inputs.

```
lab3 > six.py -
1 #write a python program for even or odd classification with validation.
2 def classify_even_odd(number):
3     if not isinstance(number, int):
4         return "Invalid input: Please enter an integer."
5     elif number % 2 == 0:
6         return "Even"
7     else:
8         return "Odd"
9 # Test the function with multiple inputs
10 test_numbers = [10, 7, -4, 0, 3.5, 'a']
11 for num in test_numbers:
12     result = classify_even_odd(num)
13     print(f"{num}: {result}")
14
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python 3.13.9 (Microsoft Store) Go Live

Ln 14, Col 5 Spaces: 4 UTF-8 CRLF Python 3.13.9 (Microsoft Store) Go Live

28°C Sunny 14:06 19-01-2026

Analysis:

Examples make the task easy to understand.

The AI gives clear even or odd results.

Input validation improves with examples.

Negative numbers are handled properly.

Wrong inputs are easier to detect.